Functional Requirements

Tower Defence Game

Jonas Leipner

November 2, 2026

1 Introduction

This document translates the project's initial user stories into functional requirements. The requirements are organised around three personas:

- **Player:** the person who plays the tower defence game, places towers, and responds to incoming enemies.
- **Researcher:** the person who analyses player behaviour and uses telemetry to investigate player strategies.
- **System Architecture:** the architectural responsibilities concerned with extracting game state and executing game logic dynamically.

2 Requirement Identification Scheme

Each functional requirement has an identifier in the form `FR-XN`. The identifier is structured as follows:

- `FR` identifies the item as a functional requirement.
- `X` identifies the MoSCoW priority: `M` means Must Have, `S` means Should Have, `C` means Could Have, and `W` means Won't Have.
- `N` is the sequential number of the requirement within the project backlog.

For example, `FR-M2` means the second functional requirement assigned the Must Have priority. The identifier is used to refer to the requirement consistently across the requirements document, implementation tasks, tests, and future Scrum sprint planning.

3 Must Have Requirements

3.1 Player Persona

User story FR-M1: As a player, I want to select towers and place a maximum of five on the grid so that I can defend my home base from incoming monsters.

Acceptance criteria

- Given that fewer than five towers have been placed, when the player selects a valid tower position, then the selected tower is placed on the grid.
- Given that five towers have been placed, when the player attempts to place another tower, then the placement is rejected and the number of placed towers remains five.
- Given that the player selects an invalid or occupied position, when they attempt to place a tower, then the placement is rejected and no existing tower is moved or removed.

Rationale: This requirement defines the basic game loop. Without tower placement and a limit on the number of towers, there is no meaningful defensive gameplay.

3.2 System Architecture Persona

User story FR-M2: As the system architecture, I want to extract the current combat state from Flame and pass it to a `dart_eval` sandbox so that game logic can be executed dynamically without needing an app restart or a new AOT compilation.

Acceptance criteria

- Given an active Flame combat session, when the dynamic evaluation step is triggered, then the current combat state is serialised into the agreed state representation and passed to the `dart_eval` sandbox.
- Given a valid combat state and evaluable game-logic code, when the sandbox executes, then it returns a result that can be applied to the active game session.
- Given that the game is already running, when new evaluable logic is supplied, then the logic can be executed without restarting the application or producing a new AOT compilation.

Rationale: This requirement is the central architectural experiment of the project. The game may be playable without dynamic evaluation, but the project objective depends on demonstrating this Flame-to-`dart_eval` boundary.

4 Should Have Requirements

4.1 Player Persona

User story FR-S1: As a player, I want enemies to be worth different point values so that I can track my score and measure my success as the waves get progressively harder.

Acceptance criteria

- Given two enemy types with different configured values, when the player defeats them, then the score increases by the corresponding value for each enemy.
- Given a completed wave, when the player views the game status, then the current score is displayed.
- Given that waves become progressively harder, when the player defeats enemies in later waves, then the score continues to reflect the configured values of those enemies.

Rationale: Different enemy values improve feedback, enjoyment, and progression. However, the core defensive game loop can function without a fully developed scoring system.

4.2 Researcher Persona

User story FR-S2: As a researcher, I want to capture telemetry data on which towers are placed most frequently so that I can analyse optimal player strategies and eventually power a dynamic recommender system.

Acceptance criteria

- Given that a player places a tower, when the placement succeeds, then a local telemetry event records the tower type and placement event.
- Given that multiple placements have been recorded, when telemetry is queried, then the system can report the placement count for each tower type.
- Given that the game is played without a network connection, when tower placements occur, then the telemetry is retained locally rather than being silently discarded.

Rationale: Telemetry supports the research goals but is not required for the game to be playable. The first implementation should therefore remain lightweight, such as a local structure recording tower-placement events.

5 Could Have Requirements

5.1 Player Persona

User story FR-C1: As a player, I want to merge two towers in the incubator to create a unique tower so that I can discover new defences and achieve a higher score.

Acceptance criteria

- Given two compatible towers in the incubator, when the player confirms the merge, then the two input towers are removed and the defined unique tower is created.
- Given two incompatible towers, when the player attempts to merge them, then the merge is rejected and both input towers remain unchanged.
- Given a successful merge, when the resulting tower is displayed, then the player can identify its type and use it as a defensive tower.

Rationale: Tower merging could provide a distinctive gameplay mechanic, but it introduces additional algorithmic, interface, balancing, and testing complexity. It is therefore outside the immediate minimum viable product.

5.2 System Architecture Persona

User story FR-C2: As the system architecture, I want to log player choices and victory metrics locally so that the data can later be formatted and fed into a dynamic recommender system.

Acceptance criteria

- Given that the player makes a tracked choice, when the choice is completed, then a local log entry records the choice and its timestamp.
- Given that a game ends, when the player wins or loses, then the local log records the outcome and the relevant victory metrics.

- Given a collection of local log entries, when the data is exported, then it is represented in a consistent format that can be processed by a future recommender system.

Rationale: This requirement is a possible future research direction rather than a fixed part of the initial project scope. It can be considered after the core architecture and gameplay loop have been demonstrated.

6 Won't Have Requirements

No requirements have currently been assigned to the Won't Have category. Requirements placed in this category would be explicitly excluded from the current project scope and would not be scheduled into the planned increments.